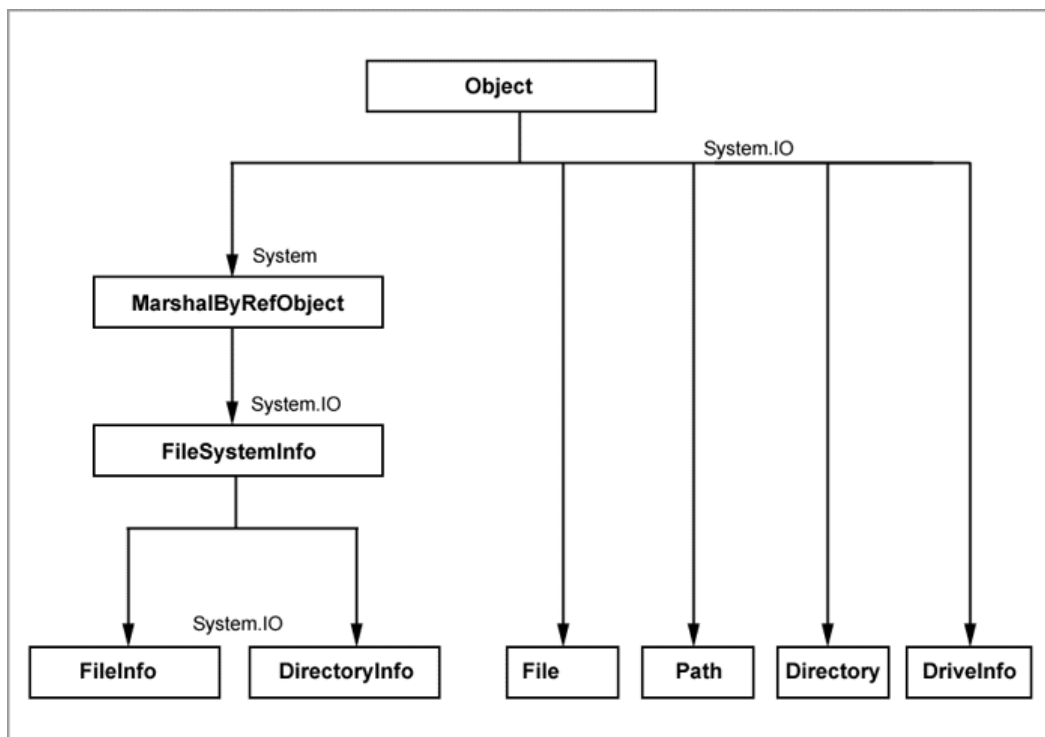


You'll learn how to use files and folders related classes available in .NET using C#.

Stream

- When you open a file for reading or writing, it becomes stream. Stream is a sequence of bytes traveling from a source to a destination over a communication path.
- The two basic streams are input and output streams. Input stream is used to read and output stream is used to write.
- The System.IO namespace includes various classes for file handling.

Diagram to represent file-handling class hierarchy



The following table describes some commonly used classes in the System.IO namespace

Class Name	Description
FileStream	It is used to read from and write to any location within a file
BinaryReader	It is used to read primitive data types from a binary stream
BinaryWriter	It is used to write primitive data types in binary format
TextReader	It is used to read characters from a byte Stream
TextWriter	It is used to write characters to a stream.
StreamReader	It is used to read from a string buffer
StreamWriter	It is used to write into a string buffer
DirectoryInfo	It is used to perform operations on directories
FileInfo	It is used to perform operations on files

A.Reading and writing in the text file

StreamWriter Class

- The **StreamWriter** class is inherited from the abstract class **TextWriter**. The TextWriter class represents a writer, which can write a series of characters.
- The following table describes some of the methods used by StreamWriter class.

Methods	Description
Close	Closes the current StreamWriter object and the underlying stream
Flush	Clears all buffers for the current writer and causes any buffered data to be written to the underlying stream
Write	Writes to the stream
WriteLine	Writes data specified by the overloaded parameters, followed by end of line

Program to write user input to a file using StreamWriter Class

```
using System;
using System.Text;
using System.IO;
namespace FileWriting_SW
{
    class Program
    {
        class FileWrite
        {
            public void WriteData()
            {
                FileStream fs = new FileStream("D:\\test.txt", FileMode.Append,
                FileAccess.Write); //File Auto Create by Append if exists then append text only

                StreamWriter sw = new StreamWriter(fs);
                Console.WriteLine("Enter the text which you want to write to the file");
                string str = Console.ReadLine();
                sw.WriteLine(str);
                sw.Flush();
                sw.Close();
                fs.Close();
            }
        }
        static void Main(string[] args)
        {
            FileWrite wr = new FileWrite();
            wr.WriteData();
        }
    }
}
```

StreamReader Class

The **StreamReader** class is inherited from the abstract class **TextReader**. The **TextReader** class represents a reader, which can read series of characters.

The following table describes some methods of the **StreamReader** class.

Methods	Description
Close	Closes the object of StreamReader class and the underlying stream, and release any system resources associated with the reader
Peek	Returns the next available character but doesn't consume it
Read	Reads the next character or the next set of characters from the stream
ReadLine	Reads a line of characters from the current stream and returns data as a string
Seek	Allows the read/write position to be moved to any position with the file

Program to read from a file using StreamReader Class

```
using System;
using System.IO;
namespace FileReading_SR
{
    class Program
    {
        class FileRead
        {
            public void ReadData()
            {
                FileStream fs = new FileStream("D:\\test.txt", FileMode.Open,
                FileAccess.Read);
                StreamReader sr = new StreamReader(fs);
                Console.WriteLine("Program to show content of test file");
                sr.BaseStream.Seek(0, SeekOrigin.Begin);
                string str = sr.ReadLine();
                while (str != null)
                {
                    Console.WriteLine(str);
                    str = sr.ReadLine();
                }
                Console.ReadLine();
                sr.Close();
                fs.Close();
            }
        }
        static void Main(string[] args)
        {
            FileRead wr = new FileRead();
            wr.ReadData();
        }
    }
}
```

The important point is that `Seek()` works with bytes, not line numbers

Hello C#

The approximate byte positions are:

Character: H e l l o C #

Position: 0 1 2 3 4 5 6 7

Peek Demo

Assume file `test.txt` contains:

HELLO

Code logic

```
StreamReader sr = new StreamReader("c:\\test.txt");
```

```
int p = sr.Peek();           // peeks next character  
char c1 = (char)p;
```

```
Console.WriteLine(c1); // Output?
```

Output

H

👉 **Pointer is still at H, not moved.**

B.Working With FileInfo In C#

Introduction

This article will explain the FileInfo class and its use. The System.IO namespace is one of the most significant namespaces used for working with Files in the .Net Framework. Let us see about the class.

FileInfo Class

The FileInfo class does not have static methods and can only be used on instantiated objects. The FileInfo object represents a file on a disk or network location. It also provides instance methods for the creation, copying, deletion, moving, and opening of files, and aids in the creation of FileStream objects. Some of the most useful methods of the FileInfo class are,

List of Methods of FileInfo Class, I have included only a few methods of the FileInfo Class.

1	Create	This method is used to create the new file.
2	CreateText	This Method Creates a StreamWriter that writes a new text file.
3	Delete	This method is used to delete a existing file.
4	CopyTo	The CopyTo method is used to copy the existing file into new file.
5	MoveTo	The MoveTo method is used to move the file one place to another valid location.
6	AppendText	This method creates a StreamWriter that appends text to the file represented by this instance of the FileInfo.
7	OpenText	This method creates a StreamReader with UTF8 encoding that reads from an existing text file.

The methods of the FileInfo class.

FileInfo.Create

This method is used to create the new file.

```
static void Main(string[] args) {  
    string path = @"D:\MyTestFile1.txt";  
    FileInfo fi = new FileInfo(path);  
    fi.Create(); {  
        Console.WriteLine("File has been created");  
    }  
}
```

FileInfo.CreateText

This Method creates a StreamWriter that writes a new text file.

```
FileInfo fi = new FileInfo(@"D:\MyTestFilecreatetext1.txt");  
StreamWriter str = fi.CreateText();  
str.WriteLine("hello");  
Console.WriteLine("File has been created with text");  
str.Close();
```

FileInfo.Delete

This method is used to delete an existing file.

```
FileInfo fi = new FileInfo(@"D:\MyTestFilecreatetext.txt");  
fi.Delete();  
Console.WriteLine("File has been deleted");
```

FileInfo.CopyTo

The CopyTo method is used to copy an existing file into a new file.

```
string path = @"d:\MyTestFile7.txt";  
string path2 = @"d:\NewFile.txt";  
FileInfo fi1 = new FileInfo(path);  
FileInfo fi2 = new FileInfo(path2);  
fi1.CopyTo(path2);  
Console.WriteLine("{0} was copied to {1}.", path, path2);
```

FileInfo.MoveTo

The MoveTo method is used to move the file from one place to another valid location.

```
string path = @ "d:\NewFile1.txt";  
string path2 = @ "E:\NewFile1.txt";  
FileInfo fi1 = new FileInfo(path);  
FileInfo fi2 = new FileInfo(path2);  
fi1.MoveTo(path2);  
Console.WriteLine("{0} was copied to {1}.", path, path2);
```

FileInfo.AppendText

This method creates a StreamWriter that appends text to the file represented by this instance of the FileInfo.

```
FileInfo fi = new FileInfo(@ "E:\NewFile1.txt");  
StreamWriter sw = fi.AppendText();  
sw.WriteLine("This");  
sw.WriteLine("is Extra");  
sw.WriteLine("Text");  
Console.WriteLine("File has been appended");  
sw.Close();
```

FileInfo.OpenText

This method creates a StreamReader with UTF8 encoding that reads from an existing text file.

```
FileInfo fi = new FileInfo(@ "E:\NewFile1.txt");  
StreamReader sr = fi.OpenText();  
string s = "";  
while ((s = sr.ReadLine()) != null) {  
    Console.WriteLine(s);  
}
```

FileInfo Code Example

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.IO;
namespace fileinfoclass1 {
    class Program {
        static void Main(string[] args) {
            // Create Method
            FileInfo fi = new FileInfo(@"D:\MyTestFile7.txt");
            FileStream fs = fi.Create();
            Console.WriteLine("File has been created");
            //CreateText Method
            FileInfo fi = new FileInfo(@"D:\MyTestFilecreatetext1.txt");
            StreamWriter str = fi.CreateText();
            str.WriteLine("hello");
            Console.WriteLine("File has been created with text");
            str.Close();
            //Delete Method
            FileInfo fi = new FileInfo(@"D:\MyTestFilecreatetext.txt");
            fi.Delete();
            Console.WriteLine("File has been deleted");
            //CopyTo Method
            string path = @"d:\MyTestFile7.txt";
            string path2 = @"d:\NewFile.txt";
            FileInfo fi1 = new FileInfo(path);
            FileInfo fi2 = new FileInfo(path2);
            fi1.CopyTo(path2);
            Console.WriteLine("{0} was copied to {1}.", path, path2);
            // MoveTo Method
            string path = @"d:\NewFile1.txt";
            string path2 = @"E:\NewFile1.txt";
            FileInfo fi1 = new FileInfo(path);
            FileInfo fi2 = new FileInfo(path2);
            fi1.MoveTo(path2);
            Console.WriteLine("{0} was moved to {1}.", path, path2);
            //ApendText Method
            FileInfo fi = new FileInfo(@"E:\NewFile1.txt");
            StreamWriter sw = fi.AppendText();
            sw.WriteLine("This");
            sw.WriteLine("is Extra");
            sw.WriteLine("Text");
        }
    }
}
```

```
Console.WriteLine("File has been appended");
sw.Close();
// Opentext Method
FileInfo fi = new FileInfo(@"E:\NewFile1.txt");
StreamReader sr = fi.OpenText();
string s = "";
while ((s = sr.ReadLine()) != null) {
    Console.WriteLine(s);
}
}
```

The FileInfo class provides the following properties that enable you to retrieve information about a file.

1	CreationTime	This property returns the creation of the file date and time.
2	Exists	The Exists property checks for the presence of a file before operating on it.
3	Extension	It returns the type of the file in the extension name
4	FullName	It returns the full name of the file from the root directory.
5	Name	The Name retrieves the name of a file.
6	LastWriteTime	It returns the last file saving date and time
7	Length	The Length retrieves the size of a file.

FileInfo Properties

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.IO;
namespace fileinfoclass1 {
    class Program {
        static void Main(string[] args) {
            FileInfo fi = new FileInfo(@"E:\NewFile1.txt");
            Console.WriteLine("File name is {0} ", fi.Name);
            Console.WriteLine("File creation time is {0} ",
fi.CreationTime.ToLongTimeString());
            Console.WriteLine("File Lastaccesstime is {0} ",
fi.LastAccessTime.ToLongDateString());
            Console.WriteLine("File length is {0} ", fi.Length.ToString() + "
Bytes");
            Console.WriteLine("File extension is {0} ", fi.Extension);
            Console.WriteLine("File exist is: ", fi.Exists);
            Console.WriteLine("File LastWriteTime is {0} ", fi.LastWriteTime);
        }
    }
}
```

Conclusion

The FileInfo class is very useful for working with files. If there is any mistake in this article then please notify me.

Directory Creation & Deletion

For Example:

```
Directory.CreateDirectory(@"Hi");
File.Delete(@".\Hello\test.txt");
Directory.Delete("Hello");
File.Copy("test.txt", "test2.txt");
File.Move("test2.txt", "test3.txt");
```

Sample Program Example

//FileTest.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.IO;

namespace ConsoleAppDemo
{
    class FileTest
    {
        public static void WriteData()
        {
            FileStream fs = new FileStream("test.txt", FileMode.Append,
FileAccess.Write);
            StreamWriter sw = new StreamWriter(fs);
            Console.WriteLine("Enter the text which you want to write to
the file");
            string str = Console.ReadLine();
            sw.WriteLine(str);
            sw.Close();
            fs.Close();
        }
        public static void ReadData()
        {
            FileStream fs = new FileStream("test.txt", FileMode.Open,
FileAccess.Read);
            StreamReader sr = new StreamReader(fs);
            Console.WriteLine("Program to show content of test file");
            sr.BaseStream.Seek(0, SeekOrigin.Begin);
            string str = sr.ReadLine();
            while (str != null)
            {
```

```

        Console.WriteLine(str);
        str = sr.ReadLine();
    }

    sr.Close();
    fs.Close();
}

public static void ReadData2()
{
    Console.WriteLine(File.ReadAllText("test.txt"));
}

public static void ReadData3()
{
    foreach(string str in File.ReadLines ("test.txt") )
    {
        Console.WriteLine(str);
    }
}

public static void WriteData2()
{
    File.WriteAllText ("test.txt","Hello Clarent Students! \n
Hello Students ");
}

public static void CreateDir(string dirName)
{
    Directory.CreateDirectory(dirName);
}

public static void RemoveDir(string dirName)
{
    Directory.Delete (dirName);
}

public static void FileRemove()
{
    File.Delete("test.txt");
}

public static void FileCopy()
{
    File.Copy("test.txt", "test2.txt");
}

public static void FileRename()
{
    File.Move("test2.txt", "text3.txt");
}

public static void ListFile(string path)
{
    string[] files = Directory.GetFiles(path);
    foreach ( string file in files)
    {

```

```
        Console.WriteLine(" : "+file);
    }
    string[] dirs = Directory.GetDirectories(path);
    foreach (string file in dirs)
    {
        Console.WriteLine(" : " + file);
    }
}
}
```